SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA37 **COURSE NAME: Software Testing**

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow. *(BL3)*

Assessment Tool Weightage: 30%

Duration :60 Mins
On Class
QUESTIONS: 5 **Total Marks:50**

Annexure A

Reverse Engineering Task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Reverse Engineering Task

Q1. Reverse Engineering of a Black-Box Web Application Using Selenium

Task Title: Reverse Engineering and Test Case Design for a Web-Based Login System

Tools: Selenium WebDriver, Chrome DevTools, Postman (optional), Excel or Google Sheets

A university uses an existing student portal for login, attendance checking, internal mark viewing, and course registration. The original requirements document and design specifications are unavailable. Students must reverse engineer the application's observable behavior and develop a systematic test suite without assuming access to its internal implementation.

The application contains the following functions:
- Username and password authentication.
- Login validation and error-message display.
- Session creation after successful login.
- Logout and session termination.
- Protection against unauthorized access to internal pages.

Practical tasks

a) Application reconnaissance and reverse engineering

1. Explore the application's available pages, forms, buttons, links, input fields, and navigation paths.
2. Use Chrome DevTools to inspect HTML elements, attributes, network requests, and browser-console errors.
3. Prepare a functional model showing the relationships between the login page, dashboard, attendance page, marks page, and logout function.
4. Derive a preliminary set of functional requirements from the observable behavior.
5. Clearly distinguish verified observations from assumptions about the hidden implementation.

b) Black-box test design

1. Identify valid and invalid equivalence classes for usernames and passwords.
2. Apply Boundary Value Analysis to any documented input-length restrictions.
3. Design positive, negative, and error-handling test cases.
4. Use random testing to select additional input combinations and identify unexpected behavior.
5. Create a test-case table containing Test Case ID, Preconditions, Input, Steps, Expected Result, Actual Result, Status, and Defect ID.

c) Automation using Selenium

1. Automate valid login, invalid login, empty-field validation, logout, and unauthorized-page access tests.
2. Use suitable element locators and explicit waits.
3. Capture screenshots when tests fail.
4. Store the test execution results in a structured report.

d) State and session testing

Construct a state transition diagram for Logged Out, Login in Progress, Logged In, and Session Expired states. Test valid and invalid transitions, including browser navigation after logout and session timeout where supported.

e) Test adequacy evaluation

Calculate the percentage of planned test cases executed and passed. Identify untested functions, prioritize risks, and explain whether the available evidence is sufficient to conclude that the application is ready for release.

Expected deliverables: Functional model, test scenarios, test-case spreadsheet, Selenium scripts, execution screenshots, defect report, state diagram, and test adequacy report.

Q2. Reverse Engineering of an Existing Java Application Using JaCoCo

Task Title: White-Box Reverse Engineering and Code Coverage Analysis of a Student Result Processing Application

Tools: Java, IntelliJ IDEA or Eclipse, JUnit, JaCoCo, Maven

A university has an existing Java application that calculates student totals, averages, grades, and pass/fail results. The application source code is available, but its original design document and test cases are missing. Several incorrect results have been reported, particularly for boundary marks and invalid inputs.

Students must reconstruct the program's logic, design tests, and evaluate the adequacy of the test suite using code coverage tools.

Sample program

```
public class ResultCalculator {

   public static String calculateGrade(int mark) {

      if (mark < 0 || mark > 100) {
         return "Invalid";
      }

      if (mark >= 90) {
         return "A";
      } else if (mark >= 75) {
         return "B";
      } else if (mark >= 50) {
         return "C";
      } else {
         return "Fail";
      }
   }
}
```

Practical tasks

a) Reverse engineering and logic reconstruction
1. Analyze the program and identify its inputs, processing logic, conditions, and possible outputs.
2. Prepare a flowchart and Control Flow Graph (CFG).
3. Identify decision nodes, branches, independent execution paths, and possible exceptional inputs.
4. Derive the implied grading rules and identify assumptions that require confirmation.

b) Black-box test design
1. Apply Equivalence Class Partitioning for marks below 0, between 0 and 100, and above 100.
2. Apply Boundary Value Analysis to the grade thresholds 50, 75, and 90.
3. Prepare test cases for values such as −1, 0, 49, 50, 74, 75, 89, 90, 100, and 101.
4. Use random testing to generate additional valid and invalid mark values.
5. Compare expected and actual outputs and document any discrepancies.

c) White-box test design
1. Identify the independent paths in the program.
2. Calculate cyclomatic complexity using \(V(G)=E-N+2P\).
3. Develop JUnit test cases to exercise the important true and false branches.

4. Compare statement coverage, branch coverage, and condition coverage.
5. Explain why executing every statement does not necessarily test every decision outcome.

d) Tool-based coverage analysis

1. Configure JaCoCo in a Maven-based Java project.
2. Execute the JUnit test suite and generate the HTML coverage report.
3. Inspect instruction, line, branch, method, and complexity information available in the report.
4. Identify uncovered lines and branches.
5. Add tests to improve coverage and regenerate the report.

e) Test adequacy evaluation

Compare the initial and improved test suites. Explain whether the coverage results establish that all grading requirements are correct. Identify additional checks needed for requirement correctness, invalid input handling, and untested combinations.

Expected deliverables: Reconstructed flowchart, CFG, complexity calculation, JUnit test scripts, JaCoCo HTML report screenshots, test results, defect log, and before-and-after coverage comparison.

Q3. Reverse Engineering and API Testing of a Banking Application

Task Title: Reverse Engineering of a Banking REST API and Validation of Transaction Rules

Tools: Postman, browser developer tools, Swagger/OpenAPI (if available), Excel

A bank provides a transaction application with APIs for retrieving account balances and initiating fund transfers. The API documentation is incomplete. Testers must infer the observable request-response behavior, reconstruct the transaction workflow, and test the application's business rules.

Use an authorized training API, a mock API, or a locally hosted sample application. Do not test a real banking system without written authorization.

Assume the following intended transaction rules:

- The transfer amount must be greater than zero.
- The transfer amount must not exceed the available balance.
- The transfer must not exceed the daily transaction limit.
- An unauthenticated request must not complete a transfer.
- A failed transaction must not incorrectly reduce the balance.

Practical tasks

a) API reverse engineering

1. Identify available endpoints, HTTP methods, request parameters, response codes, and response structures.
2. Use Postman to submit valid and invalid requests to the authorized test environment.
3. Record the observed request-response pairs and infer the API's functional behavior.
4. Prepare an API interaction diagram showing authentication, balance retrieval, transfer submission, and transaction response.
5. Document any behavior that cannot be confirmed from the available interface.

b) Black-box test design

1. Apply Equivalence Class Partitioning to transaction amounts.
2. Apply Boundary Value Analysis to the minimum permitted amount, available balance, and daily limit.
3. Create decision-table tests covering authentication, balance sufficiency, and limit validation.
4. Apply random testing to valid numeric values and malformed request parameters.
5. Design negative tests for missing fields, incorrect data types, invalid account identifiers, and unsupported HTTP methods.

c) State transition and transaction testing

Model the transaction states as Pending, Successful, Failed, and Cancelled where supported by the application. Test valid transitions and investigate whether repeated requests can create duplicate transfers.

d) White-box and control-flow analysis

If source code is supplied, construct a CFG for the transaction-validation logic, identify decision paths, and calculate cyclomatic complexity. If the source code is unavailable, explain which white-box measurements cannot be established and use externally observable behavior instead.

e) Test adequacy and defect reporting

Prepare a traceability matrix linking transaction rules to test cases. Report actual outcomes, HTTP status codes, defects, severity, and reproduction steps. Evaluate the adequacy of the test suite using requirement coverage, decision-table coverage, and applicable code coverage when source code is available.

Expected deliverables: API inventory, Postman collection, request-response screenshots, decision table, state diagram, test-case sheet, defect report, and test adequacy summary.

Q4. Reverse Engineering a Commercial Off-The-Shelf (COTS) Application

Task Title: Functional, Compatibility, and Performance Testing of an Existing COTS Application

Tools: Apache JMeter, Selenium WebDriver, browser DevTools, spreadsheet software

A university procures a third-party Learning Management System (LMS) instead of developing its own. The university does not have access to the application's source code. The LMS supports login, course enrollment, assignment uploads, and result viewing.

The university wants to determine whether the purchased application meets its operational requirements before deployment.

Practical tasks

a) Reverse engineering of observable functionality

1. Explore the LMS interface and identify its main functions, navigation paths, input fields, and outputs.
2. Prepare a functional decomposition diagram of the LMS.
3. Derive testable requirements from approved specifications and observed behavior.
4. Identify assumptions, limitations, and behaviors requiring clarification from the vendor.

b) Black-box test design

1. Design equivalence classes and boundary tests for file size, file type, login fields, and enrollment inputs.
2. Use decision tables to test assignment submission rules.
3. Apply state transition testing to submission states such as Draft, Submitted, Late, and Rejected where supported.
4. Use pairwise testing for supported browser, operating system, and device combinations.
5. Apply exploratory testing to identify usability and navigation defects.

c) Tool-based functional and performance testing

1. Automate selected LMS workflows using Selenium.
2. Use JMeter against an authorized test environment to simulate a defined workload on supported HTTP endpoints.
3. Measure response times, throughput, and error rates.
4. Compare the observed results against predefined acceptance criteria.
5. Capture screenshots and export test reports.

d) COTS testing limitations

Explain why the lack of source-code access limits CFG construction, internal path coverage, and direct JaCoCo-style instrumentation. Identify alternative evidence, including requirements-based testing, API testing, interface testing, vendor documentation, and acceptance testing.

e) Test adequacy evaluation

Develop a weighted test coverage matrix for functionality, compatibility, security, performance, and usability. Identify untested risks and recommend whether the evidence supports acceptance, conditional acceptance, or rejection against the defined criteria.
Expected deliverables: Functional decomposition diagram, compatibility matrix, Selenium scripts, JMeter results, test-case spreadsheet, defect log, screenshots, and COTS acceptance recommendation.

Q5. Reverse Engineering a Python Application Using Pytest and Coverage.py

Task Title: Reverse Engineering, Path Testing, and Test Adequacy Evaluation of a Result Validation Program

Tools: Python, pytest, Coverage.py, Pylint or Ruff

A department receives an existing Python program that validates assessment marks and calculates pass/fail status. The original requirements and test suite are unavailable. The department suspects incorrect behavior for invalid inputs, boundary values, and combinations of marks.

Students must reconstruct the program's logic, design a comprehensive test suite, and use code-analysis tools to assess its quality.

Sample program

```
def validate_result(mark, attendance):
    if mark < 0 or mark > 100:
        return "Invalid Mark"

    if attendance < 0 or attendance > 100:
        return "Invalid Attendance"

    if attendance < 75:
        return "Attendance Shortage"

    if mark >= 50:
        return "Pass"

    return "Fail"
```

Practical tasks

a) Reverse engineering and specification recovery
1. Analyze the function's inputs, conditions, outputs, and decision order.
2. Prepare a flowchart and CFG.
3. Derive the functional rules implied by the program.
4. Identify missing requirements, such as whether exactly 75% attendance qualifies and how invalid data should be reported.
5. Separate confirmed behavior from requirements that need stakeholder approval.

b) Black-box test design
1. Apply Equivalence Class Partitioning to mark and attendance inputs.
2. Apply Boundary Value Analysis at 0, 49, 50, 74, 75, 100, and relevant out-of-range values.
3. Create a decision table covering valid marks, valid attendance, attendance eligibility, and pass/fail outcomes.
4. Use random testing to generate additional input pairs.
5. Use error guessing to investigate negative values, values exceeding 100, missing inputs, and incorrect data types.

c) White-box test design and automation

1. Identify the decision nodes and independent paths.
2. Calculate cyclomatic complexity from the CFG.
3. Write pytest tests for the major decision outcomes and boundary conditions.
4. Generate a Coverage.py report and identify uncovered statements and branches where branch measurement is enabled.
5. Add test cases to exercise the missing branches.

d) Additional white-box testing

1. Identify definitions and uses of mark and attendance.
2. Explain how data-flow testing can help identify incorrect or missing variable checks.
3. Propose mutations such as changing attendance < 75 to attendance <= 75, or mark >= 50 to mark > 50.
4. Explain which test cases should fail if the test suite effectively detects each mutation.

e) Evaluate test adequacy

Compare the initial and improved test suites using statement coverage, branch coverage, boundary coverage, and requirements traceability. Explain why a high coverage percentage alone cannot prove that the program is correct.

Expected deliverables: Reverse-engineered specification, flowchart, CFG, complexity calculation, pytest scripts, Coverage.py reports, mutation-test analysis, defect log, and final adequacy evaluation.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

SIMATS ENGINEERING

CSA37 — SOFTWARE TESTING

ASSESSMENT TOOL 2

REVERSE ENGINEERING TASK — ANSWER DOCUMENT

Q1. Reverse Engineering of a Black-Box Web Application Using Selenium

1.1 Reconnaissance and recovered functional requirements

ID	Preliminary requirement	Evidence status
FR-01	The portal accepts a username and password on the login page.	Expected from brief; verify in browser
FR-02	Valid credentials lead to a dashboard and create an authenticated session.	Expected from brief; implementation not inspected
FR-03	Invalid credentials display an error and do not create an authenticated session.	Expected from brief; verify message and session
FR-04	Authenticated users can navigate to attendance and marks pages.	Expected from brief; page access not inspected
FR-05	Logout terminates the session.	Expected from brief; verify server-side behavior
FR-06	Direct access to internal pages without authentication is blocked or redirected.	Expected from brief; verify each protected route

Reconnaissance procedure:

1. Open the authorized portal and record each visible page, field, button, link, and navigation path.
2. Use Chrome DevTools → Elements to inspect labels, input types, IDs, names, form actions, and validation attributes.
3. Use Network to inspect request methods, URLs, status codes, redirects, cookies, and response behavior. Do not expose real credentials or tokens in the report.
4. Use Console to record JavaScript errors. Check whether protected pages remain inaccessible after logout and browser Back navigation.
5. Separate directly observed behavior from assumptions. Hidden backend code, password hashing, and server-side session implementation cannot be confirmed from the UI alone.

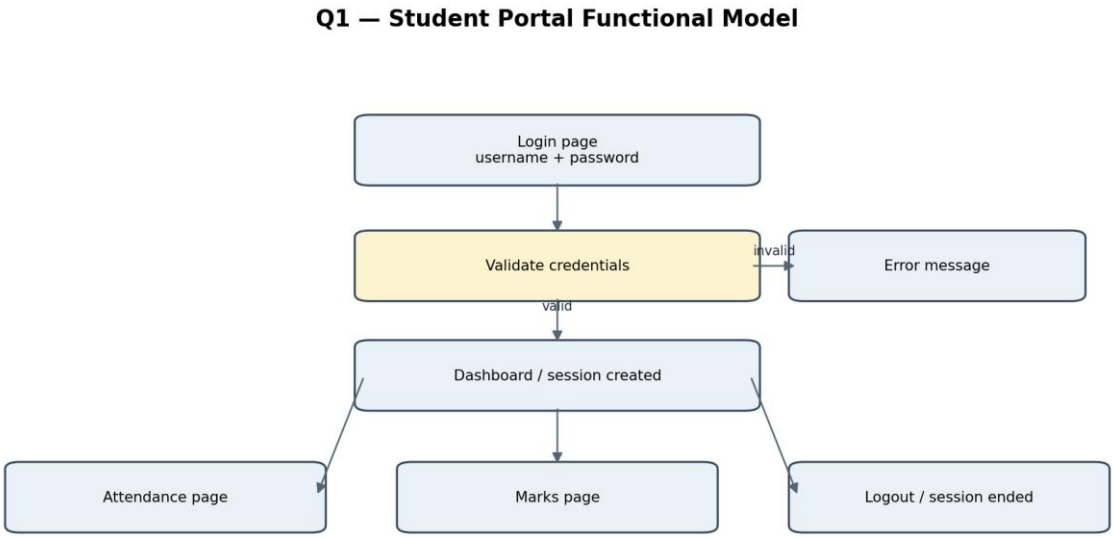


Figure 1. Reconstructed functional model; confirm all links and routes against the actual portal.

1.2 Black-box test design

Equivalence classes: (1) non-empty valid username + correct password, (2) valid username + wrong password, (3) unknown username, (4) blank username, (5) blank password, (6) both blank, (7) malformed or over-length inputs, and (8) expired or absent session. Exact length boundaries are unknown until the portal's validation attributes or approved requirements are inspected.

TC ID	Preconditions	Input	Steps	Expected result	Actual result / Status	Defect ID
L-01	Logged out	Valid username/password	Submit login	Dashboard opens; authenticated session exists	Not Executed / Not Run	—
L-02	Logged out	Valid username, wrong password	Submit login	Error shown; remain logged out	Not Executed / Not Run	—
L-03	Logged out	Unknown username + password	Submit login	Generic authentication failure; no session	Not Executed / Not Run	—
L-04	Logged out	Username blank; password filled	Submit login	Required-field validation; no session	Not Executed / Not Run	—
L-05	Logged out	Username filled; password blank	Submit login	Required-field validation; no session	Not Executed / Not Run	—
L-06	Logged out	Both fields blank	Submit login	Validation shown; no session	Not Executed / Not Run	—
L-07	Logged in	Click Logout	Click logout; revisit dashboard URL	Session ended; protected page blocked	Not Executed / Not Run	—
L-08	Logged out	Direct URL to attendance/marks	Paste protected URL	Redirect/login or access denied	Not Executed / Not Run	—
L-09	Logged in then logged out	Browser Back	Logout, then Back/refresh	Sensitive page data not accessible	Not Executed / Not Run	—
L-10	Logged out	Random strings / symbols / long values	Submit several generated combinations	No crash; safe validation response	Not Executed / Not Run	—

Boundary Value Analysis: determine the actual minimum/maximum lengths first; then test min−1, min, min+1 and max−1, max, max+1. Do not invent length limits if they are undocumented.

1.3 Selenium automation template

Install: pip install selenium. Set PORTAL_URL and locators to values inspected in DevTools. The selectors below are examples, not verified portal selectors.

```
import os
import unittest
from selenium import webdriver
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

PORTAL_URL = os.getenv("PORTAL_URL", "http://localhost:8000/login")
USER = (By.ID, "username")       # replace after DevTools inspection
PASSWORD = (By.ID, "password")   # replace after DevTools inspection
SUBMIT = (By.CSS_SELECTOR, "button[type='submit']")
ERROR = (By.CSS_SELECTOR, ".error-message")  # replace after inspection

class PortalLoginTests(unittest.TestCase):
    def setUp(self):
        options = webdriver.ChromeOptions()
        options.add_argument("--headless=new")
        self.driver = webdriver.Chrome(options=options)
        self.wait = WebDriverWait(self.driver, 10)
        self.driver.get(PORTAL_URL)

    def tearDown(self):
        self.driver.quit()

    def test_empty_fields(self):
        self.driver.find_element(*SUBMIT).click()
        # Either browser-required validation or application error is acceptable.
        self.assertTrue(
            self.driver.find_element(*USER).get_attribute("required") is not None
            or self.driver.find_elements(*ERROR)
        )

    def test_invalid_login(self):
        self.driver.find_element(*USER).send_keys("invalid_user")
        self.driver.find_element(*PASSWORD).send_keys("wrong_password")
        self.driver.find_element(*SUBMIT).click()
        self.wait.until(EC.visibility_of_element_located(ERROR))
        self.assertTrue(self.driver.find_element(*ERROR).is_displayed())

    def test_logout_and_protected_page(self):
        # Add verified successful-login steps using authorized test credentials.
        # Then click the real logout control and revisit a protected URL.
        self.skipTest("Requires portal-specific valid test credentials and selectors")

if __name__ == "__main__":
    unittest.main()
```

Failure screenshot pattern to add to a test: self.driver.save_screenshot('failure.png') inside an exception handler or test failure hook. Keep credentials in environment variables, not source code.

1.4 State transitions and adequacy

Current state	Event / condition	Next state	Expected control
Logged Out	Submit credentials	Login in Progress	Disable duplicate submit if appropriate
Login in Progress	Valid credentials	Logged In	Dashboard/session created
Login in Progress	Invalid credentials	Logged Out	Error shown; no authenticated session
Logged In	Logout	Logged Out	Session invalidated
Logged In	Timeout / expiry	Session Expired	Re-authentication required
Session Expired	Open protected URL	Logged Out / login page	No protected content exposed

Adequacy formula: execution percentage = executed test cases ÷ planned test cases × 100. Pass percentage = passed test cases ÷ executed test cases × 100. Since no portal was available, execution and pass percentages are not calculable honestly; record the counts after running the tests.

Release view: current evidence is insufficient for release approval because the application has not been executed, selectors are unverified, session timeout behavior is unknown, and no security or compatibility results exist.

Q2. Reverse Engineering of an Existing Java Application Using JaCoCo

2.1 Reconstructed logic

```
public class ResultCalculator {
    public static String calculateGrade(int mark) {
        if (mark < 0 || mark > 100) return "Invalid";
        if (mark >= 90) return "A";
        else if (mark >= 75) return "B";
        else if (mark >= 50) return "C";
        else return "Fail";
    }
}
```

Input range	Output	Recovered rule
mark < 0	Invalid	Out-of-range lower input
0–49	Fail	Valid mark below 50
50–74	C	50 is inclusive
75–89	B	75 is inclusive
90–100	A	90 and 100 are inclusive
mark > 100	Invalid	Out-of-range upper input

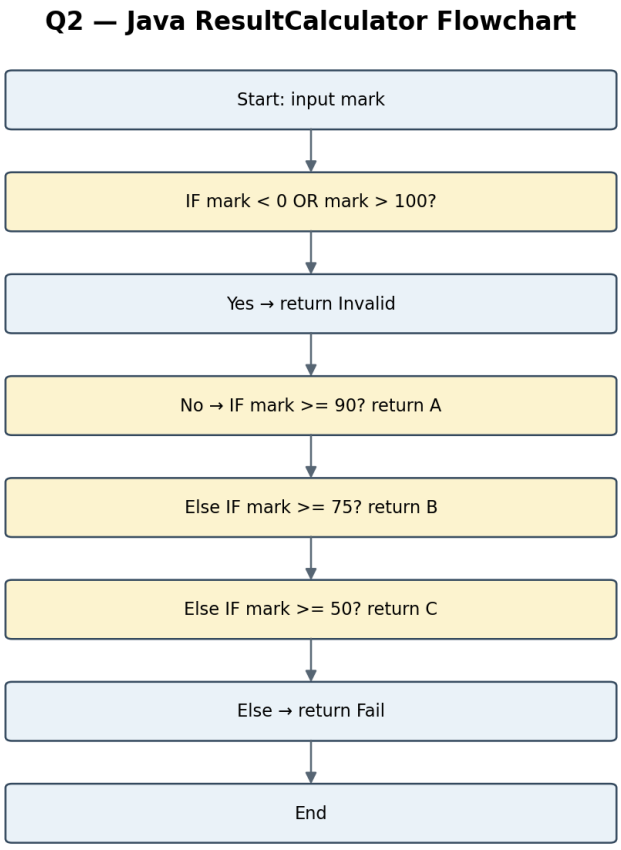


Figure 2. Flowchart reconstructed from the supplied Java code.

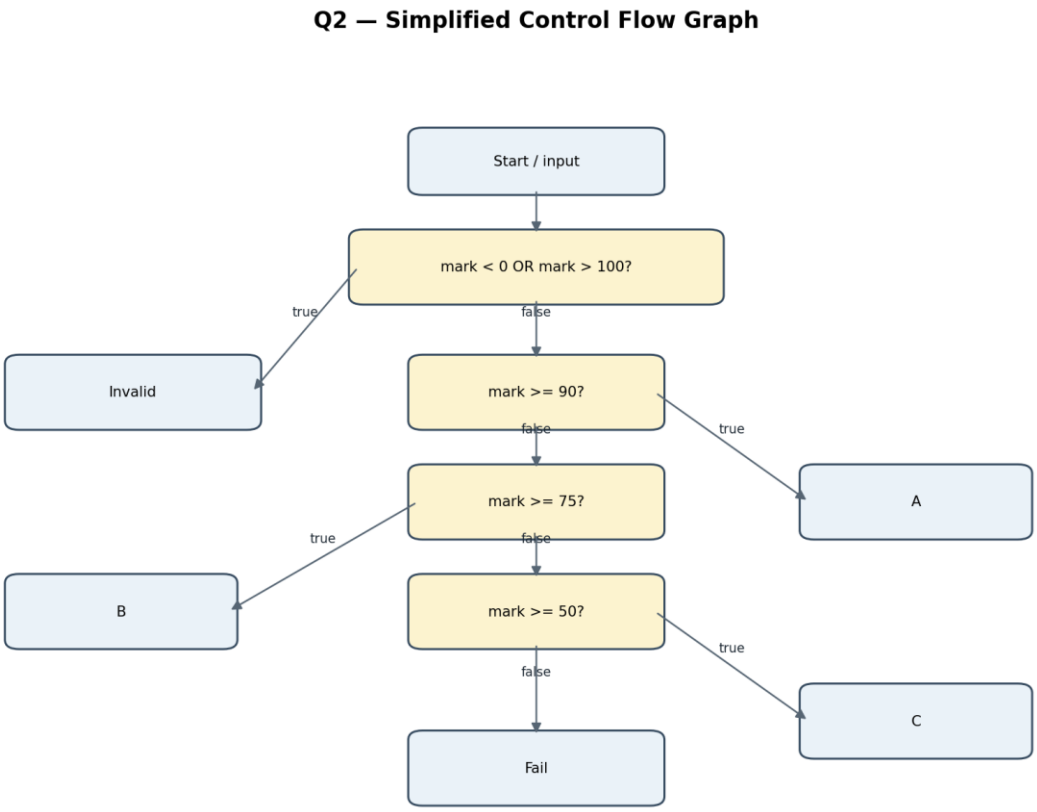


Figure 3. Simplified control flow graph (CFG).

2.2 Black-box test cases

TC	Mark	Expected output	Reason
J-01	-1	Invalid	Below minimum
J-02	0	Fail	Minimum valid mark
J-03	49	Fail	Just below 50
J-04	50	C	Threshold 50
J-05	74	C	Just below 75
J-06	75	B	Threshold 75
J-07	89	B	Just below 90
J-08	90	A	Threshold 90
J-09	100	A	Maximum valid mark
J-10	101	Invalid	Above maximum

Random testing: use a fixed seed so results are reproducible; generate values from −20 to 120 and compare each output with the recovered rules.

```
import random
random.seed(37)
for mark in random.sample(range(-20, 121), 15):
    print(mark, ResultCalculator.calculateGrade(mark))
```

2.3 Cyclomatic complexity and coverage

There are four binary decision nodes: range validation, mark ≥ 90, mark ≥ 75, and mark ≥ 50. For one connected component, V(G) = number of decisions + 1 = 4 + 1 = 5. The CFG formula V(G) = E − N + 2P gives the same result when edges/nodes are counted consistently and P = 1.

- Independent paths (basis set): invalid mark; valid mark ≥90; 75–89; 50–74; 0–49.
- Statement coverage means each executable statement is executed at least once.
- Branch coverage requires each decision outcome (true and false) to be taken.

- Condition coverage checks each atomic Boolean condition as true and false. The invalid check contains two conditions: mark < 0 and mark > 100.
- 100% statement coverage does not guarantee all decision outcomes: a test can execute a decision statement while exercising only one branch.

2.4 JUnit tests and Maven/JaCoCo

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class ResultCalculatorTest {
    @Test void boundariesAndGrades() {
        assertEquals("Invalid", ResultCalculator.calculateGrade(-1));
        assertEquals("Fail", ResultCalculator.calculateGrade(0));
        assertEquals("Fail", ResultCalculator.calculateGrade(49));
        assertEquals("C", ResultCalculator.calculateGrade(50));
        assertEquals("C", ResultCalculator.calculateGrade(74));
        assertEquals("B", ResultCalculator.calculateGrade(75));
        assertEquals("B", ResultCalculator.calculateGrade(89));
        assertEquals("A", ResultCalculator.calculateGrade(90));
        assertEquals("A", ResultCalculator.calculateGrade(100));
        assertEquals("Invalid", ResultCalculator.calculateGrade(101));
    }
}
```

Add JUnit 5 and JaCoCo to the Maven project. Example JaCoCo plugin configuration:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.jacoco</groupId>
      <artifactId>jacoco-maven-plugin</artifactId>
      <version>0.8.12</version>
      <executions>
        <execution><goals><goal>prepare-agent</goal></goals></execution>
        <execution>
          <id>report</id><phase>test</phase>
          <goals><goal>report</goal></goals>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>

mvn clean test

Open target/site/jacoco/index.html
```

Expected output after successful execution: Maven reports the number of tests run and failures; JaCoCo creates an HTML report with instruction, line, branch, method, and complexity counters. The actual percentages depend on the executed project and must be copied from the generated report. No JaCoCo report was generated in this answer environment.

2.5 Defect log and adequacy

Potential defect / risk	Severity	Evidence / next action
Incorrect threshold operator (e.g., mark > 50 instead of >= 50)	High	Boundary test at 50 detects it
Range validation omitted	High	Tests at −1 and 101 detect it
Requirement ambiguity for grades	Medium	Confirm grading bands with academic owner
Coverage interpreted as proof of correctness	Medium	Add requirements traceability, mutation tests, and invalid-input tests

Adequacy conclusion: the supplied boundary suite exercises all grade bands and both range-error directions. It is strong for the visible function, but does not prove the university’s grading policy is correct. Mutation testing and stakeholder confirmation are still required.

Q3. Reverse Engineering and API Testing of a Banking Application

3.1 API inventory and safe test setup

Use only a local mock or an explicitly authorized training API. No real banking system was contacted. Because the assessment does not provide endpoint URLs or an API schema, the inventory below is a proposed contract to adapt after inspecting the authorized environment.

Proposed endpoint	Method	Purpose	Typical outcomes to verify
/auth/login	POST	Authenticate and obtain token	200/201 success; 401 invalid credentials
/accounts/{id}/balance	GET	Retrieve available balance	200 success; 401/403 auth; 404 unknown account
/transfers	POST	Submit transfer	201 success; 400 invalid amount; 401 unauthenticated; 409/422 rule failure

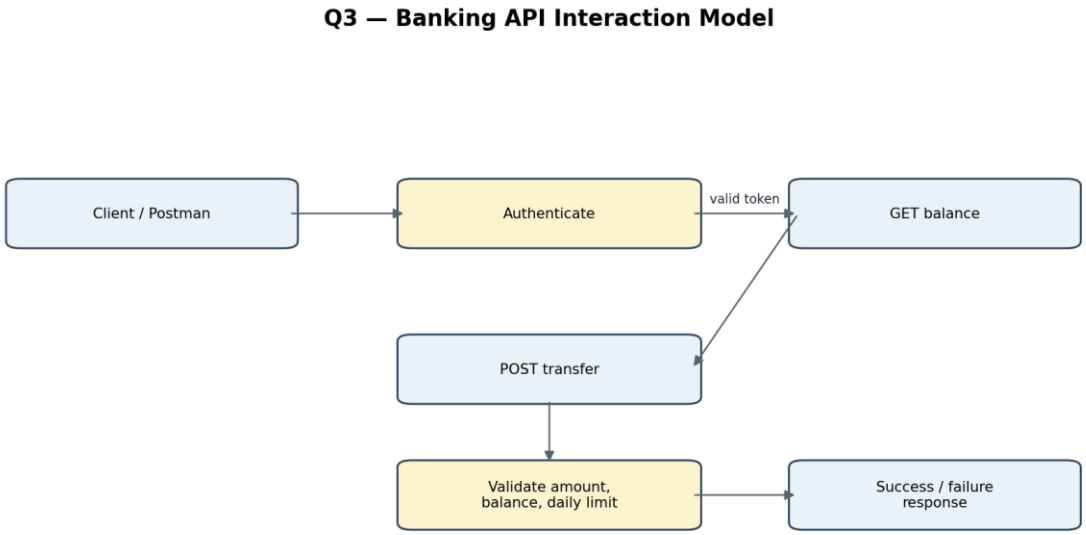


Figure 4. Proposed API interaction model; endpoint names are illustrative.

3.2 Decision table

Authenticated?	Amount > 0?	Balance sufficient?	Daily limit OK?	Expected result
No	Yes	Yes	Yes	Reject; no transfer; balance unchanged
Yes	No	Yes	Yes	Reject invalid amount
Yes	Yes	No	Yes	Reject insufficient funds
Yes	Yes	Yes	No	Reject daily limit exceeded
Yes	Yes	Yes	Yes	Accept; debit source and credit destination

3.3 Test cases and state testing

TC	Scenario / input	Expected result	Actual / status
API-01	Authenticated; positive amount below balance and limit	Success; balance updated exactly once	Not Executed
API-02	Amount = 0	Rejected; no balance change	Not Executed
API-03	Negative amount	Rejected; no balance change	Not Executed
API-04	Amount > available balance	Rejected; no balance change	Not Executed
API-05	Amount > daily limit	Rejected; no balance change	Not Executed
API-06	No authentication token	401/403; no transfer	Not Executed
API-07	Missing amount / wrong type	4xx validation response; no balance change	Not Executed
API-08	Invalid account ID / unsupported method	Appropriate 4xx response	Not Executed
API-09	Repeat same request / idempotency key	No unintended duplicate debit	Not Executed

Transaction states: Pending → Successful when all validations and commit succeed; Pending → Failed on validation or processing error; Pending → Cancelled when cancellation is supported before completion. A completed transfer should not be silently cancelled without a compensating transaction.

White-box limitation: without source code, a CFG, independent paths, and cyclomatic complexity cannot be calculated reliably. Use observable status codes, response bodies, balance checks before/after, and request logs instead.

3.4 Postman test examples

Example request body for an authorized mock service (adapt to its schema):

```
POST /transfers
Authorization: Bearer <TEST_TOKEN>
Content-Type: application/json

{
  "fromAccount": "TEST-1001",
  "toAccount": "TEST-2002",
  "amount": 25.00,
  "idempotencyKey": "test-transfer-0001"
}
```

Postman Tests tab example for a successful mock response:

```
pm.test("Response is successful", function () {
  pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});
pm.test("Response has transaction identifier", function () {
  const body = pm.response.json();
  pm.expect(body).to.have.property("transactionId");
});
```

3.5 Traceability and adequacy

Rule	Linked test cases	Evidence required
Amount > 0	API-02, API-03	Request/response and unchanged balance
Amount ≤ available balance	API-04	Balance before and after
Amount ≤ daily limit	API-05	Limit setup and rejection response
Authentication required	API-06	Unauthenticated request rejected
Failure does not debit balance	API-02 to API-08	Before/after balance comparison
No duplicate transfer	API-09	Repeated request results and ledger entries

No actual HTTP status codes, screenshots, or defects can be reported until requests are run against the authorized mock. Do not invent response evidence.

Q4. Reverse Engineering a COTS Learning Management System

4.1 Functional decomposition

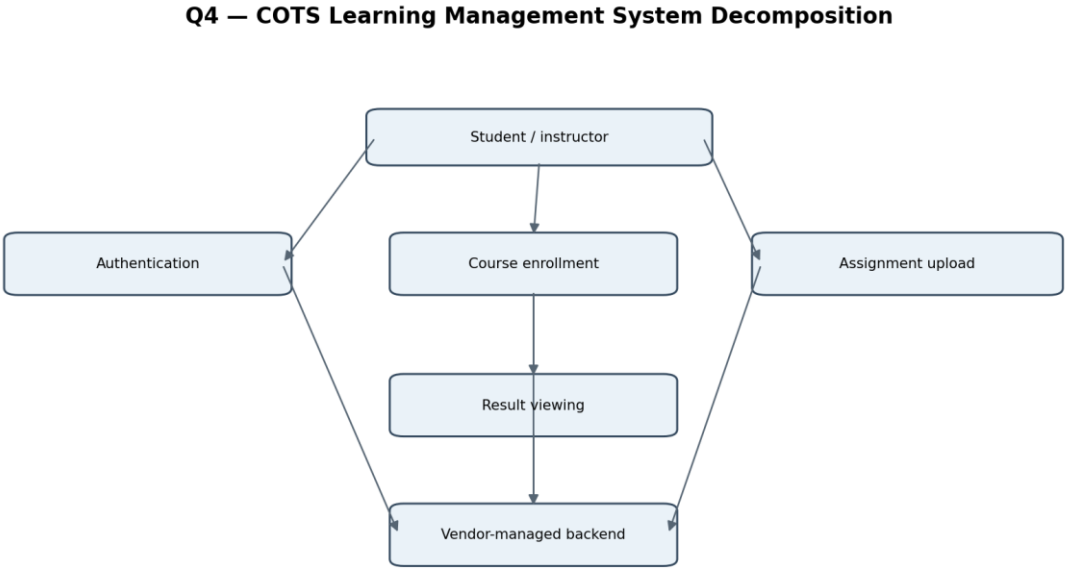


Figure 5. LMS functional decomposition based on the assessment brief.

Component	Inputs	Expected behavior / output	Risks to verify
Login	Credentials	Authenticated user session	Invalid login, session timeout, access control
Course enrollment	Course ID, student ID	Enrollment confirmation or rejection	Duplicate enrollment, capacity, invalid course
Assignment upload	File, metadata, deadline	Submission receipt and status	File size/type, network failure, late submission
Result viewing	Authenticated user, course	Correct result shown to authorized student	Wrong-user data exposure, stale results

4.2 Black-box test design

Area	Test partitions / boundaries	Expected checks
Login fields	Blank, valid, invalid, minimum/maximum length	Correct message, no unauthorized access
File type	Allowed extension, disallowed extension, renamed file	Server validates actual content, not only filename
File size	Below max, exactly max, above max	Boundary accepted; oversized file rejected safely
Enrollment	Valid course, invalid ID, duplicate, full course	Correct confirmation or clear rejection
Assignment state	Draft, Submitted, Late, Rejected (if supported)	Only allowed state transitions occur
Compatibility	Supported browser × OS × device combinations	Critical workflows behave consistently

Pairwise compatibility plan: list vendor-supported browsers, operating systems, and device types; use a pairwise tool to select combinations so every pair of factor values appears at least once. Do not claim coverage of unsupported combinations.

4.3 Selenium and JMeter procedure

6. Automate one critical workflow with Selenium: login → enroll → upload assignment → view result → logout.
7. Use explicit waits for visible status messages and page transitions. Keep test accounts and test data separate from production.
8. In JMeter, configure Thread Group, HTTP Request defaults, authorized test endpoint, assertions, and Summary/Aggregate Report listeners.

9. Run a defined baseline (for example, 5 virtual users for 1 minute) only if the vendor-approved test environment and workload permit it.
10. Record response-time percentiles, throughput, error rate, test duration, and acceptance thresholds agreed before execution.

Illustrative JMeter result format only — the numbers below are synthetic examples, not measurements from a real LMS.

Metric	Illustrative value	Interpretation
Virtual users	5	Configured workload example
Average response time	420 ms	Example only; compare to agreed target
95th percentile	900 ms	Example only; tail latency
Throughput	18 requests/sec	Example only
Error rate	0.5%	Example only; inspect each failed request

4.4 COTS testing limitations and weighted adequacy

Without source code, testers cannot directly build a reliable internal CFG, measure all internal paths, or instrument the application with JaCoCo. Alternative evidence includes requirements-based black-box testing, interface/API testing, browser DevTools, vendor documentation, security checks within authorization, and acceptance testing.

Quality area	Weight	Evidence / planned checks	Status
Functionality	35%	Login, enrollment, upload, result viewing	Not Executed
Compatibility	20%	Pairwise supported browser/OS/device matrix	Not Executed
Security	20%	Authorization, session handling, upload validation	Not Executed
Performance	15%	JMeter response time, throughput, error rate	Not Executed
Usability	10%	Exploratory navigation and error clarity	Not Executed
Total	100%	Weighted acceptance score after evidence is collected	Not calculable

Recommendation: conditional acceptance is not justified yet because there are no real execution results. Keep the system in evaluation until critical workflows, access control, file validation, compatibility, and performance criteria have been tested and signed off.

Q5. Reverse Engineering a Python Application Using Pytest and Coverage.py

5.1 Reconstructed specification

```
def validate_result(mark, attendance):
    if mark < 0 or mark > 100:
        return "Invalid Mark"
    if attendance < 0 or attendance > 100:
        return "Invalid Attendance"
    if attendance < 75:
        return "Attendance Shortage"
    if mark >= 50:
        return "Pass"
    return "Fail"
```

Priority / rule	Recovered behavior	Assumption requiring confirmation
1	Mark outside 0–100 returns Invalid Mark	Are numeric strings, None, NaN, and non-numeric values supported?
2	Attendance outside 0–100 returns Invalid Attendance	Confirm validation order when both values are invalid
3	Attendance below 75 returns Attendance Shortage	Exactly 75 qualifies by current code
4	Attendance ≥75 and mark ≥50 returns Pass	Pass threshold is inclusive
5	Attendance ≥75 and mark <50 returns Fail	Confirm grading policy with department

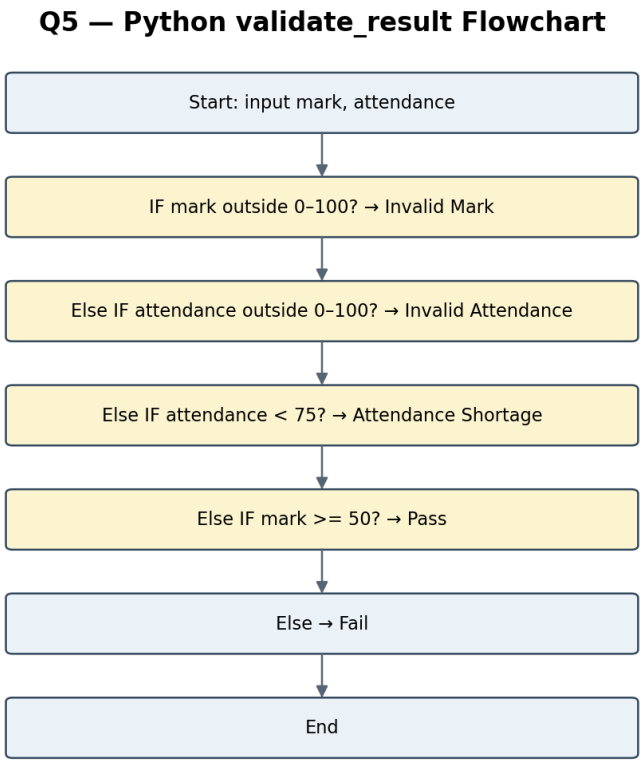


Figure 6. Flowchart reconstructed from the supplied Python function.

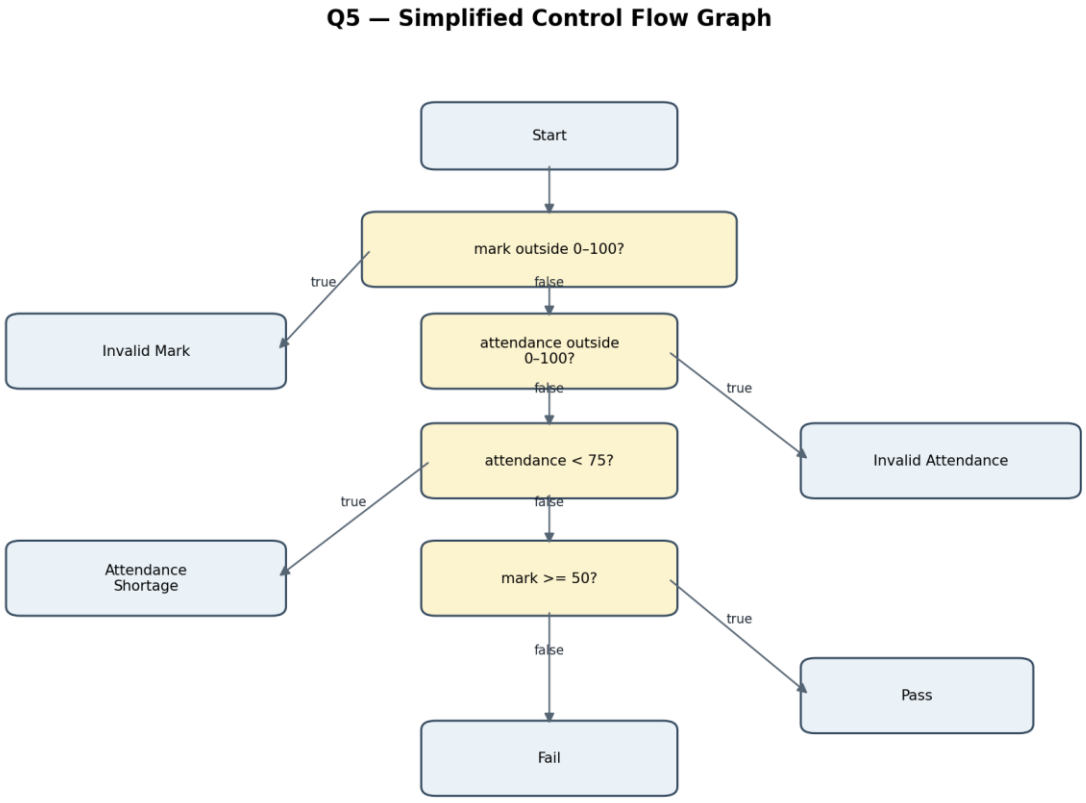


Figure 7. Simplified CFG for branch/path analysis.

5.2 Decision table and test cases

Mark valid?	Attendance valid?	Attendance ≥75?	Mark ≥50?	Expected output
No	Any	Any	Any	Invalid Mark
Yes	No	Any	Any	Invalid Attendance
Yes	Yes	No	Any	Attendance Shortage
Yes	Yes	Yes	Yes	Pass
Yes	Yes	Yes	No	Fail

TC	Mark	Attendance	Expected output	Purpose
P-01	-1	80	Invalid Mark	Lower mark boundary
P-02	101	80	Invalid Mark	Upper mark boundary
P-03	50	75	Pass	Both inclusive thresholds
P-04	49	75	Fail	Below pass threshold
P-05	80	74	Attendance Shortage	Below attendance threshold
P-06	80	75	Pass	Attendance threshold
P-07	80	0	Attendance Shortage	Minimum valid attendance
P-08	80	101	Invalid Attendance	Above attendance range
P-09	-1	101	Invalid Mark	Decision order: mark checked first

5.3 Pytest automation and coverage

```
# test_result.py
import pytest
from result import validate_result

@pytest.mark.parametrize("mark,attendance,expected", [
    (-1, 80, "Invalid Mark"),
    (101, 80, "Invalid Mark"),
    (0, 75, "Fail"),
    (49, 75, "Fail"),
    (50, 75, "Pass"),
    (100, 100, "Pass"),
```

```
    (80, 74, "Attendance Shortage"),
    (80, 0, "Attendance Shortage"),
    (80, 75, "Pass"),
    (80, 101, "Invalid Attendance"),
    (-1, 101, "Invalid Mark"),
])
def test_validate_result(mark, attendance, expected):
    assert validate_result(mark, attendance) == expected
```

Run the tests and generate statement/branch coverage:

```
pip install pytest coverage

pytest -q

coverage run --branch -m pytest

coverage report -m

coverage html

Open htmlcov/index.html
```

Expected outcome: pytest reports passed/failed tests; Coverage.py reports statement and branch coverage and identifies uncovered lines/branches. Actual percentages must be taken from your run. They are not fabricated here.

5.4 Complexity, data flow, and mutation testing

There are four binary decisions: mark range, attendance range, attendance < 75, and mark ≥ 50. Cyclomatic complexity V(G) = decisions + 1 = 5 for one connected component. A practical basis set covers invalid mark, invalid attendance, attendance shortage, pass, and fail.

- Definitions: mark and attendance are received as function parameters; the function reads them in range checks and outcome decisions.
- Data-flow testing checks whether each input is validated before use and whether validation order creates unintended behavior.
- Mutation 1: change attendance < 75 to attendance <= 75. Test (mark=80, attendance=75) must change from Pass to Attendance Shortage and should fail against the mutated program.
- Mutation 2: change mark >= 50 to mark > 50. Test (mark=50, attendance=75) must change from Pass to Fail and should detect the mutation.
- Mutation 3: remove mark range validation. Tests with mark=-1 and mark=101 should detect incorrect outputs.
- Error guessing: test None, strings, floating-point values, missing arguments, and values outside 0–100. The current function may raise TypeError for non-numeric inputs; that behavior needs an explicit requirement and tests.

5.5 Test adequacy evaluation

Measure	How to calculate / evaluate	Conclusion
Statement coverage	Executed statements ÷ executable statements × 100	Measure with Coverage.py
Branch coverage	Executed branches ÷ total branches × 100	Enable --branch
Boundary coverage	Test 0, 49, 50, 74, 75, 100 plus out-of-range values	The listed suite covers key thresholds
Requirements traceability	Every recovered rule linked to one or more tests	Maintain a rule-to-test mapping
Mutation adequacy	Mutants detected ÷ mutants run × 100	Run a mutation tool or perform controlled mutations

Final conclusion: high coverage only shows that code was executed; it does not prove the rules are correct, the requirements are complete, or all data combinations are safe. Boundary tests, mutation tests, input-type tests, and stakeholder approval are necessary.

Overall Conclusion

The five tasks demonstrate complementary reverse-engineering approaches: observable behavior and Selenium for a black-box web portal; control-flow reconstruction and JaCoCo for Java; request-response and business-rule testing for an API; requirements-based acceptance testing for a source-unavailable COTS LMS; and pytest/Coverage.py with mutation analysis for Python.

Submission checklist:

- Replace all placeholders and confirm requirements against the actual application or authorized mock.
- Run Selenium, JUnit/JaCoCo, Postman, JMeter, and pytest/Coverage.py where the assessment environment permits.
- Insert authentic screenshots and reports from your own runs; do not submit illustrative metrics as actual evidence.
- Complete Actual Result, Status, and Defect ID columns after execution.
- Use the generated diagrams as reconstructed models, then correct them if observations differ.

Appendix: Tool commands quick reference

Tool	Command / action
Selenium	pip install selenium; python test_portal.py
Maven + JUnit + JaCoCo	mvn clean test; open target/site/jacoco/index.html
Postman	Import authorized collection; set environment variables; run collection
JMeter	jmeter; configure Thread Group and HTTP Request; run only on approved test target
pytest + Coverage.py	pytest -q; coverage run --branch -m pytest; coverage report -m; coverage html